

Reddit Clone

Microservices Backend

How to Use & Run — Step-by-Step Guide

Node.js · TypeScript · Express · PostgreSQL · Redis · RabbitMQ · Docker Compose

1. What you are running

This is a Reddit-style social platform split into **eight microservices** behind a single **API Gateway**. Each service owns its own PostgreSQL database, services talk to each other asynchronously through **RabbitMQ** events, and the feed is ranked in **Redis**. Everything runs with one command via Docker Compose.

The gateway is the only public entry point — all requests go to **http://localhost:8080**.

Service	Purpose	Database
api-gateway	Routing, JWT validation, rate limiting	—
auth-service	Register / login / refresh, JWT issuance	auth_db
user-service	Profiles & karma	user_db
community-service	Communities, join / leave	community_db
post-service	Posts	post_db
comment-service	Nested comments	comment_db
vote-service	Up / down votes	vote_db
feed-service	Hot & top ranking	Redis
notification-service	Event-based notifications	notification_db

2. Prerequisites

You only need **Docker Desktop** (which includes Docker Compose v2). No local Node.js is required — every service builds and runs inside its own container.

Requirement	Notes
Docker 20+ & Compose v2	Use <code>docker compose</code> (space), not the old <code>docker -compose</code> .
Free ports	8080 (API), 5432 (Postgres), 6379 (Redis), 5672 & 15672 (RabbitMQ).
~2 GB free RAM	For the 9 app containers + 3 infrastructure containers.

3. Step 1 — Configure the environment

From the project root, copy the environment template to `.env`:

```
# macOS / Linux
cp .env.example .env

# Windows PowerShell
Copy-Item .env.example .env
```

The defaults work out of the box for local use. For anything beyond local testing, change **JWT_SECRET** and the database / RabbitMQ passwords.

4. Step 2 — Build and start everything

```
docker compose up --build
```

The first run downloads base images and compiles each service — give it a few minutes. The stack is ready when every service prints a line like `listening on :4001`. Postgres, Redis, and RabbitMQ have health checks, so the app services wait for them automatically.

To run it in the background instead, use `docker compose up --build -d`.

5. Step 3 — Verify it is running

```
curl http://localhost:8080/health  
# {"status":"ok","service":"api-gateway"}
```

You can also open the **RabbitMQ management UI** at `http://localhost:15672` (default login `reddit / reddit_pass`) to watch the `reddit.events` exchange and the per-service queues.

6. Step 4 — Use the API (full walkthrough)

Every call goes through the gateway at **http://localhost:8080**. Protected endpoints need an Authorization: Bearer <token> header.

6.1 Register a user

```
curl -X POST http://localhost:8080/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"username":"alice","email":"alice@example.com","password":"password123"}'

# Response includes: user, accessToken, refreshToken
# Behind the scenes: a profile is created and a 'welcome' notification is sent.
```

6.2 Log in (later sessions)

```
curl -X POST http://localhost:8080/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"identifier":"alice","password":"password123"}'
# 'identifier' accepts either the username or the email.
```

6.3 Create a community

```
curl -X POST http://localhost:8080/communities \
  -H 'Authorization: Bearer <accessToken>' \
  -H 'Content-Type: application/json' \
  -d '{"name":"programming","description":"All things code"}'
# Returns the community id; the creator auto-joins.
```

6.4 Create a post

```
curl -X POST http://localhost:8080/posts \
  -H 'Authorization: Bearer <accessToken>' \
  -H 'Content-Type: application/json' \
  -d '{"communityId":"<communityId>","title":"Hello world","body":"My first post"}'
```

6.5 Vote on the post

```
curl -X POST http://localhost:8080/votes \
  -H 'Authorization: Bearer <accessToken>' \
  -H 'Content-Type: application/json' \
  -d '{"targetType":"post","targetId":"<postId>","value":1}'
# value: 1 = upvote, -1 = downvote, 0 = remove vote
```

6.6 Read the feed and notifications

```
# Hot feed (time-decayed ranking)
curl http://localhost:8080/feed

# Your notifications
curl http://localhost:8080/notifications -H 'Authorization: Bearer <accessToken>'
```

See the README for the complete endpoint reference (comments, leaving communities, vote scores, marking notifications read, and more).

7. Everyday commands

Command	What it does
<code>docker compose up --build</code>	Build (if needed) and start the whole stack.
<code>docker compose up -d</code>	Start in the background (detached).
<code>docker compose ps</code>	Show status of all containers.
<code>docker compose logs -f auth-service</code>	Tail the logs of one service.
<code>docker compose up -d --build post-service</code>	Rebuild and restart a single service.
<code>docker compose down</code>	Stop and remove containers.
<code>docker compose down -v</code>	Stop and also delete the Postgres data volume (full reset).

8. Troubleshooting

Symptom	Fix
502 Upstream service unavailable	The target service is still starting. Wait for its 'listening on' log or check <code>docker compose logs <svc></code> .
A service keeps restarting	It could not reach Postgres / RabbitMQ on first boot. It recovers on the automatic restart.
Port already in use	Another process holds 8080 / 5432 / 6379 / 5672. Stop it or change the port in <code>.env</code> .
Stale data after schema changes	Run <code>docker compose down -v</code> , then <code>up --build</code> .
401 Authentication required	Missing or expired token. Log in again, or call <code>/auth/refresh</code> .

9. How it fits together

When you act on the platform, services publish events that other services react to. This keeps each service independent and the system responsive:

Event	Triggered by	Reaction
<code>user.created</code>	Registering	Profile created · welcome notification
<code>post.created</code>	Creating a post	Post indexed into the ranked feed
<code>comment.created</code>	Commenting	The replied-to user is notified
<code>vote.created</code>	Voting	Score, author karma, feed rank & notification all update

Feed ranking uses the decay formula $\text{rank} = \text{score} / (\text{ageInHours} + 2)^{1.5}$, recomputed in Redis every time a vote arrives.